

Multilingual Toxicity Detection

Course: ELEC-E5550 - Statistical Natural Language Processing
D

Aapo Eronen - ID: 101034836
Pouya Amiri - ID: 102839441
Vu Minh Nguyen - ID: 101676647
Jere Malinen - ID: 907268

October 5, 2026

1 Introduction

The proliferation of online platforms has revolutionized human interaction, enabling instantaneous communication across the globe. However, these online platforms have also facilitated the spread of toxic content—ranging from hate speech and cyberbullying to harassment and misinformation. Such toxicity not only undermines the quality of online discourse but also poses significant psychological and societal risks. [1]

Detecting toxic content is a complex task due to the ambiguity of natural language. Toxicity can be overt, involving explicit language, or covert, expressed through sarcasm, context-dependent meanings, or cultural nuances. For instance, a phrase deemed innocuous in one cultural context might be offensive in another.

The significance of addressing online toxicity cannot be overstated. Exposure to toxic content has been linked to adverse mental health outcomes [2]. Moreover, the existence and amount of such content can erode trust in online platforms and discourage user engagement.

From a societal perspective, unchecked toxicity can exacerbate polarization, fuel misinformation, and hinder constructive interactions. For platform providers, the challenge lies in balancing the enforcement of community standards with the preservation of free expression. Effective toxicity detection mechanisms are thus essential for fostering safe and inclusive online environments. [2]

1.1 Approaches to Toxicity Detection

Various methodologies have been developed to detect toxic content, each with its strengths and limitations:

- **Rule-Based Systems:** These rely on predefined lists of offensive terms and patterns. While straightforward, they often fail to capture context-dependent toxicity and can be circumvented through slight alterations in language. Approaches like these might be used in user name sanitation, or e.g. offensive clan-tag removal in games.
- **Machine Learning Models:** Supervised learning techniques utilize annotated datasets to train classifiers that can identify toxic content. These models can capture complex patterns but require large, high-quality datasets and may struggle with generalization across different contexts, or languages. The baseline approach in this report utilizes an LSTM for this task.
- **Deep Learning and Transformer-Based Models:** Advanced models like BERT and GPT have shown promise in understanding nuanced language patterns. However, they can be computationally intensive and may inadvertently learn biases present in training data. The main approach in this report is based on the transformer architecture. A more precise description is given in chapter 2.1.
- **Context-Aware and Multimodal Approaches:** Recent research has utilized contextual information, such as user history or conversation threads for sentiment analysis for predicting future toxicity [3].

Despite these advancements, challenges persist. Models often struggle with the subtleties of human language, such as sarcasm or cultural references. Additionally, the dynamic nature of online communication requires continuous model updates and adaptations.

2 Methods

2.1 Baseline Model

The baseline approach is a vanilla LSTM with binary output. LSTMs are adept at capturing long-range dependencies, making them suitable for tasks like toxicity classification [4]. Unlike the main approach, the baseline only has one model trained on only English data. The model employs the XLM-RoBERTa-base tokenizer, which leverages a multilingual vocabulary and subword embeddings. This choice helps handling texts in multiple languages, such as English, German, and Finnish. Before tokenization, the text samples can be demojized (emojis are replaced with a textual description). Since the training data had an imbalance in labels, the loss function was chosen to be BCEWithLogitsLoss with a pos-weight to address this imbalance. To address the imbalance in predictions, the model outputs are thresholded using a sigmoid function which favors positive labels.

2.2 Main Model

Our toxicity detection pipeline begins with a language classification stage. We employ a lightweight LoRA-based classifier to categorize input text into one of three languages: German, English, or Finnish. Once the language is identified, the text is routed to the corresponding language-specific toxicity detection model, which is also based on a fine-tuned LoRA architecture. This modular approach allows each model to specialize in linguistic and cultural nuances relevant to its target language, improving overall detection accuracy and reducing cross-language bias.

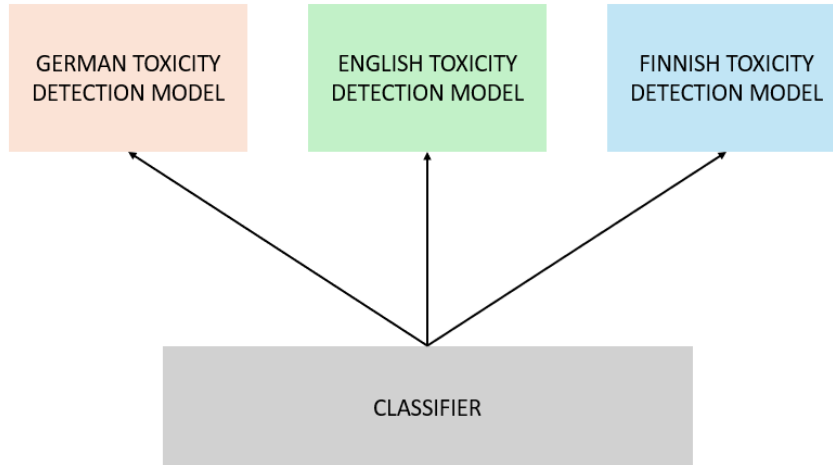


Figure 1: Pipeline Overview

3 Experiments and Results

3.1 Baseline Model

3.1.1 Preprocessing

The raw TSV file provided was split into datasets containing only one language. From these, the English and German datasets were demojized. Finnish was also demojized, however the replacements were in English since no support for Finnish was provided by the used library. No other preprocessing steps were done for the training, or evaluation sets. The test set was not demojized.

3.1.2 Tokenization

Tokenization was done with the XLM-RoBERTa-base tokenizer. Inputs are capped at 64 tokens to cover most inputs while maintaining computational efficiency.

3.1.3 Trainig Setup

Training is conducted exclusively on English data, with the expectation that the model can generalize learned patterns to other languages.

- Epochs: 20
- Batch Size: 256
- Learning Rate: Initially set to 1e-3, with dynamic adjustment via a learning rate scheduler.
- Loss Function: BCEWithLogitsLoss with a computed pos_weight to address class imbalance (63% non-toxic, 37% toxic in training set).
- Optimizer: AdamW
- Learning Rate Scheduler: ReduceLROnPlateau reduces the learning rate by a factor of 0.2 if validation F1 fails to improve for 3 consecutive epochs.

3.1.4 Experiment 1: Course Testset & Devset

The baseline model evaluation scores on the labelled devset and unlabelled devset are shown in the below tables.

Language	Accuracy	Precision	Recall	F1
ENG	0.8872	0.8126	0.8833	0.8457
FIN	0.5800	0.7946	0.5933	0.6794
GER	0.5534	0.3109	0.6428	0.4172

Table 1: Basemodel evaluation scores on the labelled devset.

Metric	Multilingual	ENG	FIN	GER
Accuracy	0.7200	0.8828	0.5613	0.5782
Precision	0.6195	0.8842	0.8168	0.3273
Recall	0.7836	0.8810	0.5988	0.6673
F1	0.6920	0.8826	0.6910	0.4392

Table 2: Basemodel scores on the unlabelled testset (as reported by Codabench).

3.2 Model for English Toxicity Detection

The English experiments involved several attempts using different models and dataset sizes. While the overall settings remained mostly consistent across runs, some adjustments were made in each attempt based on the model and dataset size to enhance performance and reduce training time. The configuration described below corresponds to the final setup used for our submission.

Data

Initially, only the provided training dataset was used. Later on, we incorporated additional data from the "Self-Detoxifying Language Models via Toxicification Reversal" experiment by Leong et al. [5]¹.

Preprocessing

The provided course data was used without any modifications.

The additional dataset, however, required preprocessing. Unlike the course data, it was not labeled with binary classes but instead included continuous scores between 0 and 1 for multiple categories such as "toxicity", "profanity", "sexually explicit", "identity attack", "flirtation", "threat", "insult", and "severe toxicity".

To convert this data to binary classification, we used the "toxicity" score: samples with values above 0.7 were labeled as toxic (1), and those below 0.3 as non-toxic (0). Samples with values in between were discarded to reduce ambiguity. After filtering, approximately 133,000 samples remained. These were used as-is in the early experiments.

For the final experiment, further preprocessing was applied:

- Unicode Normalization (NFKC)
- Trimming whitespace from both ends and collapsing extra spaces (these were mostly artifacts from converting JSONL to TSV)

After preprocessing, label distribution was roughly 30.7k toxic (1) and 102.7k non-toxic (0) samples. To address the class imbalance, non-toxic samples with fewer than 10 tokens were dropped, reducing the count to 93.7k. Then, a random 20% of the remaining non-toxic samples were removed, bringing the total to around 50k. Finally, the data was shuffled and prepared for the final round of training.

Token Length Analysis

To determine an appropriate maximum token length for model training, we wrote a Python script to analyze the token distribution across the `train`, `dev`, and `extra` datasets. The analysis used the DeBERTa-v3 tokenizer to compute the number of tokens per text input and provided summary statistics, including threshold exceedance counts.

Based on the results, we chose a maximum length of **256 tokens**, which preserves nearly all samples while providing enough capacity for longer texts. Only 0.04% of the data exceeds this length.

Metric	Value
Number of samples	247,254
Average token length	31.02
Median token length	17
Minimum token length	1
Maximum token length	1,204
90th percentile	74
95th percentile	114
99th percentile	202
Samples > 128 tokens	9,982 (4.04%)
Samples > 256 tokens	98 (0.04%)
Samples > 384 tokens	15 (0.01%)
Samples > 512 tokens	10 (0.00%)

Table 3: Token length statistics across all datasets.

¹<https://github.com/cooperleong00/ToxicificationReversal/tree/master/data>

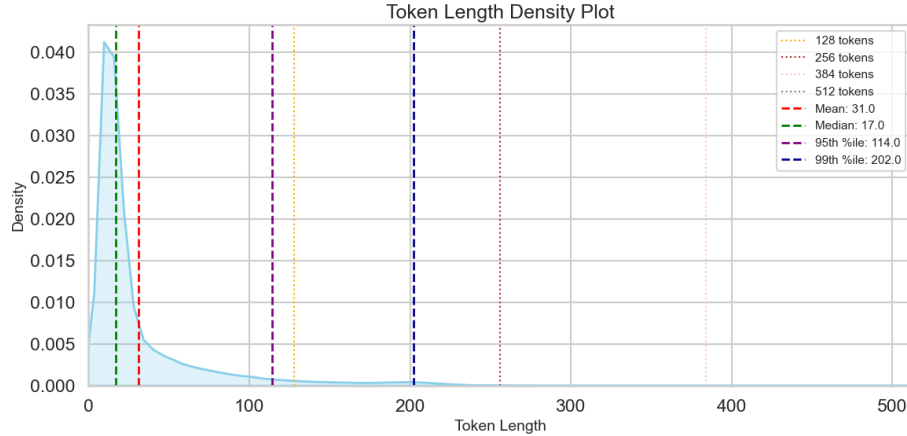


Figure 2: Token Length Analysis

Tokenization

The `AutoTokenizer` from Hugging Face was used to automatically select the appropriate tokenizer for each pretrained model. The maximum token length was set to 256 tokens to ensure compatibility with model constraints while retaining the vast majority of input samples.

Model and Training Setup

For our classification model, we either used `microsoft/deberta-v3-base` or `roberta-base` architecture. The encoder was frozen to reduce training time and memory usage, and we applied Low-Rank Adaptation (LoRA) with rank $r = 16$, $\alpha = 32$, and a dropout rate of 0.05. LoRA was applied specifically to the `query_proj`, `key_proj`, and `value_proj` modules of the transformer.

The training was performed using mixed-precision with `bfloat16`, a maximum token length of 256, and gradient accumulation over 2 steps. We trained for 7 epochs with a batch size of 16, using the AdamW optimizer with $\epsilon = 1 \times 10^{-6}$. A cosine learning rate scheduler with restarts was enabled, including a 10% warmup phase. The learning rate was set to 3×10^{-4} .

Evaluation and checkpointing were performed at the end of each epoch. For metric reporting, both accuracy and weighted F1 score were used.

Training and evaluation were conducted using the HuggingFace `Trainer` API.

3.2.1 Experiment 1: Course Trainset + RoBERTa-Base

In the first experiment, the base RoBERTa model (`roberta-base`) was fine-tuned using the provided course trainset and validated on the devset (only the English samples). The model was trained for 5 epochs. While the results were promising, this model was not submitted to Codabench as we aimed to further improve the performance in subsequent experiments.

3.2.2 Experiment 2: Course Trainset + DeBERTaV3-Base

In the second experiment, we fine-tuned the DeBERTaV3 base model (`microsoft/deberta-v3-base`) on the course trainset. This setup yielded improved performance compared to the previous RoBERTa-based model. The model was trained for 5 epochs, and the checkpoint from epoch 2 was selected for submission based on validation performance.

The checkpoint from epoch 2 was submitted to Codabench, and the final test results are reported below:

Parameter	Value
Tokenizer max length	256
Batch size	16
Epochs	7
Learning rate	3×10^{-4}
Gradient accumulation steps	2
Dropout	0.5
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
Frozen encoder	Yes
Precision	<code>bfloat16</code>
Learning rate scheduler	Cosine with restarts
Warmup steps	10% of total
Adam epsilon	1×10^{-6}
Evaluation strategy	Per epoch
Metrics	Accuracy, Weighted F1

Table 4: Summary of training configuration and hyperparameters.

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score
1	0.1806	0.1951	0.9322	0.9325
2	0.2287	0.1822	0.9389	0.9391
3	0.1179	0.1839	0.9392	0.9396
4	0.1104	0.1840	0.9383	0.9387
5	0.2477	0.1829	0.9402	0.9405

Table 5: Performance of RoBERTa-Base on the course trainset over 5 epochs.

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score
1	0.2216	0.1794	0.9361	0.9366
2	0.1641	0.1655	0.9421	0.9424
3	0.0716	0.1711	0.9424	0.9428
4	0.1201	0.1772	0.9426	0.9430
5	0.2605	0.1711	0.9438	0.9442

Table 6: Performance of DeBERTaV3-Base on the course trainset over 5 epochs.

F1 Score	Accuracy	Precision	Recall
0.93	0.93	0.90	0.97

Table 7: Submission results on Codabench using epoch 2 checkpoint.

3.2.3 Experiment 3: Course Trainset + Extra Set + DeBERTaV3-Base

In this experiment, we reused the DeBERTaV3-Base model with a similar configuration to the previous setup. However, the training data was extended by including the extra dataset, which was only lightly preprocessed. The model was trained for 5 epochs. Although the validation metrics did not show significant improvement during training, submitting the checkpoint from epoch 4 yielded noticeably better results on the test set.

The checkpoint from epoch 4 was submitted to Codabench, yielding the following performance:

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score
1	0.0901	0.1939	0.9385	0.9389
2	0.0839	0.1895	0.9388	0.9394
3	0.0580	0.1848	0.9418	0.9422
4	0.0822	0.1786	0.9424	0.9428
5	0.1180	0.1811	0.9424	0.9428

Table 8: Performance of DeBERTaV3-Base trained on the course trainset and extra set.

F1 Score	Accuracy	Precision	Recall
0.94	0.94	0.92	0.97

Table 9: Submission results on Codabench using epoch 4 checkpoint.

3.2.4 Experiment 4: Course Trainset + Extra Set + RoBERTa-Base

Following the promising results from adding the extra dataset, the RoBERTa-Base model was revisited. A similar configuration to Experiment 1 was used, but this time the training was extended to 8 epochs. The validation metrics showed steady improvement, achieving the highest scores seen so far during training.

However, despite these promising numbers, submissions using checkpoints from epochs 6 and 3 yielded disappointing results on the Codabench test set. Both showed a high rate of false positives, which could not be mitigated by threshold tuning.

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score
1	0.1265	0.1636	0.9411	0.9414
2	0.0859	0.1690	0.9398	0.9405
3	0.0977	0.1533	0.9455	0.9458
4	0.0685	0.1560	0.9447	0.9452
5	0.1210	0.1635	0.9462	0.9466
6	0.0721	0.1510	0.9476	0.9480
7	0.0660	0.1563	0.9474	0.9477
8	0.1018	0.1613	0.9468	0.9472

Table 10: Performance of RoBERTa-Base trained on the course trainset and extra set.

Checkpoint	F1 Score	Accuracy	Precision	Recall
Epoch 6	0.87	0.86	0.78	0.99
Epoch 3	0.93	0.92	0.88	0.98

Table 11: Submission results using RoBERTa-Base checkpoints.

3.2.5 Experiment 5: Course Trainset + Clean Extra Set + DeBERTaV3-Base

For the final experiment, the `microsoft/deberta-v3-base` model was used once again, this time with two key modifications. First, several hyperparameters were adjusted to improve training efficiency and model performance. Second, the extra dataset was heavily preprocessed and balanced, as detailed in the Preprocessing subsection.

The model was trained for 6 epochs. This setup led to a notable improvement in performance, with the checkpoint from epoch 4 achieving the best results on the test set.

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score
1	0.1134	0.183687	0.9359	0.9367
2	0.1211	0.157886	0.9443	0.9448
3	0.0960	0.145867	0.9498	0.9502
4	0.1047	0.139325	0.9513	0.9516
5	0.0581	0.155257	0.9492	0.9495
6	0.0324	0.161741	0.9502	0.9505

Table 12: Performance of DeBERTaV3-Base trained on the course trainset and clean extra set.

F1 Score	Accuracy	Precision	Recall
0.95	0.95	0.94	0.97

Table 13: Submission results on Codabench using epoch 4 checkpoint.

3.3 Model for German Toxicity Detection

The German toxicity detection experiment follows the same training and inference settings as the classifier, differing only in the dataset and base model used. The dataset was filtered to include only German entries, and all preprocessing and model fine-tuning were tailored accordingly.

Preprocessing

The raw TSV file provided by the course was filtered to include only entries where the `id` field begins with `ger`. The following text-cleaning steps were applied (See Appendix C):

- Only in experiment 3: Replace three or more asterisks (***) with the word *<some-bad-word>* to recover redacted toxic expressions.
- Removed all user mentions (e.g., `@username`) and URLs.
- Normalized repeated characters (e.g., `sooo` $\rightarrow$ `so`).
- Trimmed whitespace and ensured all entries were valid strings.

The processed dataset was saved for use in both training and evaluation.

Tokenization

The processed text was tokenized using a HuggingFace tokenizer with a maximum sequence length of 128. Missing or empty text entries were replaced with a blank space and assigned a default label of 0 to avoid training errors.

Model and Training Setup

The base model for this experiment was fine-tuned using Low-Rank Adaptation (LoRA) with the following configuration:

- Rank: $r = 8$
- LoRA alpha: 64
- Dropout: 0.2

The model was trained for 10 epochs with:

- Learning rate: 1×10^{-5}
- Weight decay: 1×10^{-3}

Training and evaluation were conducted using the HuggingFace `Trainer` API.

3.3.1 Experiment 1: Course Devset + XLM-RoBERTa

This experiment used the first 200 German entries from the course-provided devset as training data. The model was initialized with `cardiffnlp/twitter-xlm-roberta-large-2022` [6, 7], a multilingual transformer trained on Twitter data. LoRA was applied for fine-tuning using standard toxicity classification settings.

F1 Score	Accuracy	Precision	Recall
0.62	0.76	0.51	0.80

Table 14: Submission results on Codabench.

3.3.2 Experiment 2: textdetox/multilingual_toxicity_dataset + XLM-RoBERTa

This experiment used the German split of the `textdetox/multilingual_toxicity_dataset` [8] loaded via the HuggingFace `datasets` library. The same multilingual `twitter-xlm-roberta-large-2022` [6, 7] model was used with identical training hyperparameters and LoRA configuration.

F1 Score	Accuracy	Precision	Recall
0.63	0.80	0.58	0.69

Table 15: Submission results on Codabench.

3.3.3 Experiment 3: DeTox Dataset + BERT-German-Cased

The third experiment used the DeTox dataset [9], a comprehensive resource of 10,278 German social media comments annotated with 12 fine-grained labels covering toxicity, discrimination, and criminal relevance. We used the `google-bert/bert-base-german-cased` model for this setting, which is pre-trained specifically on German texts. LoRA fine-tuning was again applied.

In addition to the shared preprocessing pipeline, the preprocessing includes replacing sequences of three or more asterisks (***) with the word `<some-bad-word>` to recover redacted toxic expressions.

F1 Score	Accuracy	Precision	Recall
0.65	0.80	0.56	0.76

Table 16: Submission results on Codabench.

3.4 Model for Finnish Toxicity Detection

In Finnish experiments we fine-tuned the multilingual Transformer XLM-T using `cardiffnlp/twitter-xlm-roberta-large-2022` [6, 7]. We kept the architecture and principal hyperparameters mostly fixed so that we could isolate the effects of three factors: the amount of training data, the preprocessing pipeline, and the loss function.

3.4.1 Data

Depending on the experiment, we drew from up to three datasets. The first dataset was provided by the course and machine translated with google translator. We supplemented this with the machine translated JigsawToxicity dataset and the Finnish language Suomi24 forum dataset, both released by TurkuNLP [10]. Because the latter two datasets originally contained multilevel toxicity labels, we collapsed them to the binary toxic/nontoxic scheme used in the course data.

3.4.2 Preprocessing

The course corpus and Suomi24 were already relatively clean, so we removed only explicit URLs, which might introduce some unnecessary noise. The jigsaw texts, which come from Wikipedia, contained additional noise such as HTML tags and table markups. In our second experiment we left those untouched and in the last one we executed additional preprocessing. For further details we direct the reader to the Appendix.

3.4.3 General training setup

The XLM-T model was fine-tuned using Low-Rank Adaptation. We conducted experiments using following training configurations:

- Rank: $r = 16$
- LoRA alpha: 16
- LoRA Dropout: 0.05
- learning rate: 2×10^{-5}
- Weight decay: 0.01

We used HuggingFace’s standard tokenizer for the XLM-T model with maximum sequence length of 128 or 256 depending on the experiment. Number of training epochs and batch sizes varied as well between experiments from 5 to 6 and 8 to 16 respectively. For more details, see individual experiments below.

3.4.4 Experiment 1: Machine translated course dataset

In the first experiment we fine-tuned the XLM-T model using only a machine translated version of the training set provided by the course. We trained for 5 epochs using batch size of 8 and maximum sequence length of 128.

F1 Score	Accuracy	Precision	Recall
0.88	0.81	0.92	0.84

Table 17: Submission results on Codabench.

3.4.5 Experiment 2: Balanced combined dataset

In the second experiment we created a balanced iterative dataset leveraging all datasets. That was accomplished using torch’s IterableDataset class using all positive examples across all training sets and sampling same number of negative examples from the rest for each epoch.

We fine-tuned XLM-T model with resulting dataset for 6 epochs using batch size of 16 and maximum sequence length of 128.

F1 Score	Accuracy	Precision	Recall
0.91	0.84	0.90	0.91

Table 18: Submission results on Codabench.

3.4.6 Experiment 3: Combined dataset with customized loss function

For the third experiment we used all datasets like in experiment 2, but this time instead of creating smarted dataset we just pooled all the data in one regular dataset and tried to include our knowledge about unequal label distribution to the weighted cross entropy loss function. Weights were computed using inverse probabilities resulting in 0.177 and 0.823 for non-toxic and toxic respectively. Notice that additional data preprocessing for jigsaw dataset was executed for this experiment before pooling the data.

The XLM-T model was fine-tuned using the resulting dataset and loss function for 5 epochs using batch size of 16 and maximum sequence length of 256.

F1 Score	Accuracy	Precision	Recall
0.91	0.85	0.93	0.89

Table 19: Submission results on Codabench using classification threshold of 0.2.

3.5 Model for Language Classification

For language classification, we used the multilingual model `cardiffnlp/twitter-xlm-roberta-base` as the base model, fine-tuned using Low-Rank Adaptation (LoRA) to reduce training cost while maintaining performance. The classification task involves detecting the language of a given text sample—German, English, or Finnish—based on its identifier prefix (`ger`, `eng`, or `fin`), which is mapped to labels 0, 1, and 2, respectively.

We constructed a balanced development set by selecting the first 200 datapoints of each language from the devset provided by the course. The final evaluation was performed on the test set provided by the course.

The dataset was tokenized with a maximum sequence length of 128 and padded accordingly. We trained the model using the following LoRA configuration: rank $r = 8$, LoRA alpha = 64, and dropout = 0.2.

The model was trained for 10 epochs with a learning rate of 1×10^{-5} and a weight decay of 1×10^{-3} .

F1 Score	Accuracy	Precision	Recall
0.998	0.998	0.996	0.997

Table 20: Result of the classifier on the course’s testset

4 Conclusions and Discussion

In this report, we covered various approaches to multilingual toxicity detection task, highlighting a baseline LSTM model and an advanced transformer-based pipeline incorporating language-specific models. Our modular transformer approach employing LoRA fine-tuning demonstrated superior performance across languages capturing linguistic nuances relatively effectively.

Multiple experiments were conducted using different models, such as DeBERTaV3 and RoBERTa, as well as varying configurations of datasets to assess their impact on detection metrics.

Despite advancements in our experiments, certain challenges persist, for example the accurate handling of culturally specific expressions, sarcasm, and contextual toxicity. Future research could benefit from experimenting with ensemble methods, combining predictions from multiple models to potentially improve robustness and reduce false positives. Additionally, developing language-specific preprocessing and text normalization strategies tailored to unique linguistic characteristics could further enhance model accuracy across diverse cultural contexts.

5 Division of labor

5.1 Appo Eronen

- Trained the Finnish models

5.2 Pouya Amiri

- Trained the English models
- Carried out the unsuccessful experiments in the section [Appendix A: Unsuccessful Experiments](#)

5.3 Vu Minh Nguyen

- Trained the German models
- Trained the classifier

5.4 Jere Malinen

- Trained the baseline model

6 Acknowledgments

We gratefully acknowledge Prof. Dr. Melanie Siegel for providing the DeTox dataset² [9], which was used to train our German model. Although the dataset is publicly available, access must be requested via email, which she kindly granted to us.

²<https://github.com/hdasprachtechnologie/detox>

References

- [1] A. Sheth, V. L. Shalin, and U. Kursuncu, “Defining and detecting toxicity on social media: context and knowledge are key,” *Neurocomputing*, vol. 490, p. 312–318, June 2022.
- [2] M. Warner, A. Strohmayr, M. Higgs, and L. Coventry, “A critical reflection on the use of toxicity detection algorithms in proactive content moderation systems,” *International Journal of Human-Computer Studies*, vol. 198, p. 103468, Apr. 2025.
- [3] Éloi Brassard-Gourdeau and R. Khoury, “Using sentiment information for preemptive detection of toxic comments in online conversations,” 2020.
- [4] A. Kumar and P. Kumar, “Investigating bias in automatic toxic comment detection: An empirical study,” 2021.
- [5] C. T. Leong, Y. Cheng, J. Wang, J. Wang, and W. Li, “Self-detoxifying language models via toxification reversal,” 2023.
- [6] D. Loureiro, F. Barbieri, L. Neves, L. Espinosa Anke, and J. Camacho-collados, “TimeLMs: Diachronic language models from Twitter,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, (Dublin, Ireland), pp. 251–260, Association for Computational Linguistics, May 2022.
- [7] F. Barbieri, L. Espinosa Anke, and J. Camacho-Collados, “XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond,” in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, (Marseille, France), pp. 258–266, European Language Resources Association, June 2022.
- [8] D. Dementieva, D. Moskovskiy, N. Babakov, A. A. Ayele, N. Rizwan, F. Schneider, X. Wang, S. M. Yimam, D. Ustalov, E. Stakovskii, A. Smirnova, A. Elnagar, A. Mukherjee, and A. Panchenko, “Overview of the multilingual text detoxification task at pan 2024,” in *Working Notes of CLEF 2024 - Conference and Labs of the Evaluation Forum* (G. Faggioli, N. Ferro, P. Galuščáková, and A. G. S. de Herrera, eds.), CEUR-WS.org, 2024.
- [9] C. Demus, J. Pitz, M. Schütz, N. Probol, M. Siegel, and D. Labudde, “Detox: A comprehensive dataset for German offensive language and conversation analysis,” in *Proceedings of the Sixth Workshop on Online Abuse and Harms (WOAH)*, (Seattle, Washington (Hybrid)), pp. 143–153, Association for Computational Linguistics, July 2022.
- [10] A. Eskelinen, L. Silvala, F. Ginter, S. Pyysalo, and V. Laippala, “Toxicity detection in finnish using machine translation,” in *The 24rd Nordic Conference on Computational Linguistics*, 2023.

Appendix A: Unsuccessful Experiments

Throughout the development phase, numerous experiments were conducted that unfortunately did not lead to significant improvements or promising outcomes. Nevertheless, they collectively consumed over a hundred hours of training time and experimentation. We think it worth mentioning a few of these attempts below.

Experiment 1: Translating the Full English Dataset to German

Over 200,000 English samples were translated to German using the `Helsinki-NLP/opus-mt-en-de` model. However, the translation introduced various artifacts, necessitating heavy post-processing and cleanup.

The `microsoft/mdeberta-v3-base` model was then trained on this translated dataset combined with the original German samples. Training was conducted for 14 epochs, and the checkpoint from epoch 9 was submitted. Unfortunately, the results were poor, likely due to the quality of the machine-translated data.

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score
1	0.1984	0.6224	0.7455	0.7607
2	0.2062	0.6216	0.7270	0.7449
3	0.2465	0.4982	0.7810	0.7926
4	0.1448	0.4736	0.7900	0.8003
5	0.2520	0.5177	0.7825	0.7952
6	0.2110	0.4603	0.8000	0.8092
7	0.1842	0.4757	0.7980	0.8077
8	0.1572	0.4787	0.7980	0.8079
9	0.2015	0.4543	0.7945	0.8048
10	0.2074	0.4621	0.7970	0.8068
11	0.1566	0.4665	0.8020	0.8118
12	0.1696	0.4714	0.8040	0.8140
13	0.2026	0.4686	0.8040	0.8137
14	0.1892	0.4701	0.8025	0.8122

Table 21: Performance of mDeBERTaV3-Base on translated English-to-German data.

Submission (Epoch 9):

F1	Accuracy	Precision	Recall
0.58	0.42	0.95	0.66

Table 22: Submission results of Experiment 1.

Experiment 2: Pseudo-Ensembling Multiple Checkpoints

Checkpoints from epochs 9, 11, 12, and 13 were pseudo-ensembled with soft voting. However, the validation results were not promising and the experiment was discontinued.

Experiment 3: Translating the German Test Set to English

To utilize the stronger English model, the German portion of the test set was translated into English using the `facebook/nllb-200-distilled-1.3B` model. Although the translations were slightly better than those from `Helsinki-NLP/opus-mt-en-de`, the output still contained many artifacts that required heavy postprocessing.

The final English model was used to make predictions on the translated data, but the submission resulted in the worst performance observed in the project.

F1	Accuracy	Precision	Recall
0.53	0.76	0.52	0.53

Table 23: Submission results of Experiment 3.

Experiment 4: Training a Decoder-Only Model (LLaMA 3)

This experiment was performed purely out of curiosity, with no specific expectations.

The `meta-llama/Llama-3.2-3B` decoder-only model was fine-tuned on nearly 20,000 German samples (The smaller dataset were chosen due to extremely long training times) for 3 epochs. We managed to reduce the training loss to 0.0010 :)

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.5693	0.7070	0.8265	0.6959	0.6193	0.7940
2	0.3394	0.7843	0.8495	0.7464	0.6448	0.8860
3	0.0010	0.9193	0.8360	0.7368	0.6153	0.9180

Table 24: Performance of LLaMA 3 on German dataset.

Appendix B: Code and Resources

A Baseline Training

Listing 1: Baseline training code

```
1 import torch
2 from torch.utils.data import DataLoader
3 from torch.optim.lr_scheduler import ReduceLROnPlateau
4 from baseline.model import VanillaLSTM
5 from baseline.preprocess import create_trainset_raw, create_devset_raw,
6     tokenize_and_prepare
7 from baseline.config import LSTMConfig
8 from baseline.logger import save_checkpoint, log_run
9 from baseline.lstm_utils import compute_accuracy, compute_metrics
10 from collections import defaultdict, Counter
11 import os
12
13 save_model = True # set to false to not save models
14 log = True # set to false to not generate logs
15 clean = True # removes emojis and replaces them with descriptions in the
16     correct language, except finnish
17 load_pretrained = False
18 model_path = 'saved_models/best_lstm.pt'
19
20 def main():
21     cfg = LSTMConfig()
22     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
23     print(f"Using device: {device}")
24
25     print("Loading and tokenizing training data...")
26     train_ds = create_trainset_raw(multilingual=False, clean=clean)
27     print(Counter(train_ds["label"]))
28     train_ds = tokenize_and_prepare(train_ds, cfg.tokenizer, cfg.max_len)
29     train_ds.set_format(type="torch", columns=["input_ids", "attention_mask",
30         "label"])
31     train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=
32         True)
33
34     print("Loading and tokenizing dev data...")
35     dev_dict = create_devset_raw(per_language=True, clean=clean)
36     for lang in dev_dict:
37         dev_dict[lang] = tokenize_and_prepare(dev_dict[lang], cfg.tokenizer,
38             cfg.max_len)
39         dev_dict[lang].set_format(type="torch", columns=["input_ids", "
40             attention_mask", "label"])
41
42     model = VanillaLSTM(
43         vocab_size=cfg.vocab_size,
44         embedding_dim=cfg.embed_dim,
45         hidden_dim=cfg.hidden_dim,
46         output_dim=1,
47         padding_idx=cfg.pad_idx,
48     ).to(device)
49
50     pos_weight = torch.tensor([62530/36470]).to(device)
51     criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
52     optimizer = torch.optim.Adam(model.parameters(), lr=cfg.lr)
53
54     scheduler = ReduceLROnPlateau(
```

```

50     optimizer,
51     mode='max',
52     factor=0.2,
53     patience=3,
54     verbose=True,
55     threshold=1e-3
56 )
57
58 if load_pretrained and os.path.exists(model_path):
59     print("Loading checkpoint...")
60     checkpoint = torch.load(model_path, map_location=device)
61     model.load_state_dict(checkpoint["model_state_dict"])
62     optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
63     start_epoch = checkpoint.get("epoch", 0) + 1
64 else:
65     start_epoch = 0
66
67 for epoch in range(start_epoch, cfg.epochs):
68     model.train()
69     total_loss, total_acc, total_prec, total_rec, total_f1 = 0, 0, 0, 0, 0
70
71     for batch in train_loader:
72         input_ids = batch["input_ids"].to(device)
73         labels = batch["label"].float().to(device)
74
75         optimizer.zero_grad()
76         outputs = model(input_ids).squeeze()
77         loss = criterion(outputs, labels)
78         preds = (torch.sigmoid(outputs) > 0.6).float()
79         acc = compute_accuracy(preds, labels)
80         prec, rec, f1 = compute_metrics(preds, labels)
81
82         loss.backward()
83         optimizer.step()
84
85         total_loss += loss.item()
86         total_acc += acc.item()
87         total_prec += prec
88         total_rec += rec
89         total_f1 += f1
90
91     avg_loss = total_loss / len(train_loader)
92     avg_acc = total_acc / len(train_loader)
93     avg_prec = total_prec / len(train_loader)
94     avg_rec = total_rec / len(train_loader)
95     avg_f1 = total_f1 / len(train_loader)
96
97     print(f"Epoch{epoch+1}: Train Loss={avg_loss:.4f} | Acc={avg_acc:.4f} | Prec={avg_prec:.4f} | Rec={avg_rec:.4f} | F1={avg_f1:.4f}")
98
99
100 model.eval()
101 final_metrics = defaultdict(lambda: defaultdict(dict))
102 all_f1s, all_accs, all_precs, all_recs, all_losses = [], [], [], [], []
103 with torch.no_grad():
104
105     for lang, ds in dev_dict.items():
106         dev_loader = DataLoader(ds, batch_size=cfg.batch_size)
107         val_loss, val_acc, val_prec, val_rec, val_f1 = 0, 0, 0, 0, 0
108
109         for batch in dev_loader:

```

```

110         input_ids = batch["input_ids"].to(device)
111         labels = batch["label"].float().to(device)
112
113         outputs = model(input_ids).squeeze()
114         loss = criterion(outputs, labels)
115         preds = (torch.sigmoid(outputs) > 0.6).float()
116         acc = compute_accuracy(preds, labels)
117         prec, rec, f1 = compute_metrics(preds, labels)
118
119         val_loss += loss.item()
120         val_acc += acc.item()
121         val_prec += prec
122         val_rec += rec
123         val_f1 += f1
124
125         val_loss /= len(dev_loader)
126         val_acc /= len(dev_loader)
127         val_prec /= len(dev_loader)
128         val_rec /= len(dev_loader)
129         val_f1 /= len(dev_loader)
130
131         print(f"[{lang.upper()}] Validation: Loss={val_loss:.4f} | Acc={val_acc:.4f} | Prec={val_prec:.4f} | Rec={val_rec:.4f} | F1={val_f1:.4f}")
132
133         final_metrics[f"epoch_{epoch}"][f"{lang}"]["loss"] = val_loss
134         final_metrics[f"epoch_{epoch}"][f"{lang}"]["accuracy"] = val_acc
135         final_metrics[f"epoch_{epoch}"][f"{lang}"]["precision"] = val_prec
136         final_metrics[f"epoch_{epoch}"][f"{lang}"]["recall"] = val_rec
137         final_metrics[f"epoch_{epoch}"][f"{lang}"]["f1"] = val_f1
138
139         all_f1s.append(val_f1)
140         all_accs.append(val_acc)
141         all_precs.append(val_prec)
142         all_recs.append(val_rec)
143         all_losses.append(val_loss)
144
145         avg_val_f1 = sum(all_f1s) / len(all_f1s)
146         avg_val_acc = sum(all_accs) / len(all_accs)
147         avg_val_prec = sum(all_precs) / len(all_precs)
148         avg_val_rec = sum(all_recs) / len(all_recs)
149         avg_val_loss = sum(all_losses) / len(all_losses)
150
151         print(f"[AVG] Validation: Loss={avg_val_loss:.4f} | Acc={avg_val_acc:.4f} | Prec={avg_val_prec:.4f} | Rec={avg_val_rec:.4f} | F1={avg_val_f1:.4f}")
152
153         final_metrics[f"epoch_{epoch}"]["avg_validation_f1"] = avg_val_f1
154         final_metrics[f"epoch_{epoch}"]["avg_validation_accuracy"] = avg_val_acc
155         final_metrics[f"epoch_{epoch}"]["avg_validation_precision"] = avg_val_prec
156         final_metrics[f"epoch_{epoch}"]["avg_validation_recall"] = avg_val_rec
157         final_metrics[f"epoch_{epoch}"]["avg_validation_loss"] = avg_val_loss
158
159         scheduler.step(avg_val_f1)
160
161         if save_model:

```

```
162         save_checkpoint(  
163             model,  
164             optimizer,  
165             epoch,  
166             path=model_path,  
167             score=avg_val_f1,  
168             metric_name="f1"  
169         )  
170  
171     if log:  
172         final_metrics = {k: dict(v) for k, v in final_metrics.items()}  
173         log_run(cfg, final_metrics)  
174  
175 if __name__ == "__main__":  
176     main()
```

B Baseline Definition

Listing 2: Baseline model definition

```
1 import torch.nn as nn
2
3 class VanillaLSTM(nn.Module):
4     def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
5         padding_idx):
6         super(VanillaLSTM, self).__init__()
7         self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=
8             padding_idx)
9         self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True)
10        self.fc = nn.Linear(hidden_dim, output_dim)
11        #self.sigmoid = nn.Sigmoid()
12
13    def forward(self, input_ids):
14        embedded = self.embedding(input_ids)
15        _, (hidden, _) = self.lstm(embedded)
16        output = self.fc(hidden[-1])
17        #return self.sigmoid(output)
18        return output
```

C German Preprocessing

Listing 3: German Preprocessing

```
1 def process_tsv(input_path, output_path):
2     df = pd.read_csv(input_path, sep='\t')
3     df = df[df['id'].str.startswith('ger')]
4     df['text'] = df['text'].astype(str) # Ensure 'text' is a string
5     df['text'] = df['text'].str.replace(r'\s{3,}', '<some-bad-word>', regex=
        True)
6     df['text'] = df['text'].apply(lambda x: re.sub(r'@\S+', '', x))
7     df['text'] = df['text'].str.replace(r'http\S+', '', regex=True)
8     df['text'] = df['text'].str.replace(r'(\.)\1{2,}', r'\1', regex=True)
9     df['text'] = df['text'].str.strip()
10    df.to_csv(output_path, sep='\t', index=False)
```

D English Preprocessing

Listing 4: JSON to binary classified TSV

```
1 import polars as pl
2 import json
3
4 def load_jsonl(file_path):
5     with open(file_path, "r", encoding="utf-8") as f:
6         return [json.loads(line) for line in f]
7
8 def process_data(data):
9     texts = []
10    labels = []
11
12    for item in data:
13        for part in ["prompt", "continuation"]:
14            text = item[part]["text"]
15            toxicity = item[part].get("toxicity", None)
16
17            if toxicity is not None:
18                if toxicity > 0.7:
19                    label = 1
20                elif toxicity < 0.3:
21                    label = 0
22                else:
23                    continue
24
25            texts.append(text)
26            labels.append(label)
27
28    return pl.DataFrame({"text": texts, "label": labels})
29
30 def save_tsv(df, output_file):
31     df.write_csv(output_file, separator="\t")
32
33 def main(input_file, output_file):
34     data = load_jsonl(input_file)
35     df = process_data(data)
36     save_tsv(df, output_file)
37     print(f"Processed {df.shape[0]} rows and saved to {output_file}")
38
39 input_file = "rtp-train-90k.jsonl"
40 output_file = "extra.tsv"
41 main(input_file, output_file)
```

Listing 5: English Preprocessing

```
1 import polars as pl
2 import unicodedata
3 import re
4
5 extra_data = pl.read_csv("../data/extra.tsv", separator="\t", quote_char=
6     None)
7
8 def clean_text(text):
9     if text is None:
10         return text
11     text = unicodedata.normalize("NFKC", text)
12     text = text.strip()
13     text = re.sub(r"\s+", " ", text)
14     return text
```

```

14
15 extra_data = extra_data.with_columns(
16     pl.col("text").map_elements(clean_text, return_dtype=pl.Utf8)
17 )
18
19 extra_data = extra_data.with_columns(
20     pl.col("text").str.split(" ").list.len().alias("token_count")
21 ).filter(
22     (pl.col("label") == 1) | (pl.col("token_count") >= 10)
23 ).drop("token_count")
24
25 np.random.seed(42)
26
27 label_0_data = extra_data.filter(pl.col("label") == 0)
28 label_1_data = extra_data.filter(pl.col("label") == 1)
29 sample_size_0 = int(len(label_0_data) * 0.8)
30 label_0_sampled = label_0_data.sample(n=sample_size_0, with_replacement=False,
31     seed=42)
32 balanced_data = label_0_sampled.vstack(label_1_data)
33 shuffled_data = balanced_data.sample(n=len(balanced_data), shuffle=True, seed
    =42)
shuffled_data.write_csv("../data/shuffled_data.tsv", separator="\t")

```

E English DeBERTa Training

Listing 6: DeBERTaV3

```
1 from transformers import AutoModelForSequenceClassification, AutoTokenizer,
   Trainer, TrainingArguments, DataCollatorWithPadding, AutoConfig
2 from peft import LoraConfig, get_peft_model
3 import torch
4 from sklearn.metrics import f1_score, accuracy_score
5 from datasets import concatenate_datasets
6 from datasets import Dataset
7 import polars as pl
8
9 device = torch.device('cuda:0')
10 torch_dtype = torch.bfloat16
11
12 config = {
13     'model': 'microsoft/deberta-v3-base',
14     'dropout': 0.5,
15     'max_length': 256,
16     'batch_size': 16,
17     'epochs': 7,
18     'lr': 3e-4,
19     'enable_scheduler': True,
20     'scheduler': 'cosine_with_restarts',
21     'gradient_accumulation_steps': 2,
22     'adam_eps': 1e-6,
23     'freeze_encoder': True
24 }
25
26 tokenizer = AutoTokenizer.from_pretrained(config['model'], use_fast=False)
27 model = AutoModelForSequenceClassification.from_pretrained(
28     pretrained_model_name_or_path=config['model'],
29     config=AutoConfig.from_pretrained(config['model']),
30     torch_dtype = torch.bfloat16,
31 )
32 if config["freeze_encoder"]:
33     for param in model.deberta.parameters():
34         param.requires_grad = False
35
36 lora_config = LoraConfig(
37     r=16,
38     lora_alpha=32,
39     lora_dropout=0.05,
40     target_modules=['query_proj', 'key_proj', 'value_proj'],
41     bias="none",
42 )
43 peft_model = get_peft_model(model, lora_config)
44 peft_model = peft_model.to(device)
45
46 train_data = pl.read_csv("../data/train.tsv", separator="\t", quote_char=
   None)
47 dev_data = pl.read_csv("../data/dev.tsv", separator="\t", quote_char=None)
48 extra_data = pl.read_csv("../data/shuffled_data.tsv", separator="\t",
   quote_char=None)
49 trainset = Dataset.from_polars(train_data)
50 devset = Dataset.from_polars(dev_data)
51 extraset = Dataset.from_polars(extra_data)
52 devset = devset.filter(lambda example: example["id"].startswith("eng_"))
53 trainset = concatenate_datasets([trainset, extraset])
54 trainset = trainset.shuffle(seed=42)
55
```

```

56 class TokenizedDataset(Dataset):
57     def __init__(self, dataset, tokenizer):
58         self.dataset = dataset
59         self.tokenizer = tokenizer
60
61     def __len__(self):
62         return len(self.dataset)
63
64     def __getitem__(self, idx):
65         example = self.dataset[idx]
66         tokenized = self.tokenizer(
67             example["text"],
68             truncation=True,
69             max_length=config["max_length"],
70             return_tensors=None
71         )
72
73         label = example["label"]
74         if isinstance(label, list):
75             label = label[0]
76         label = int(label)
77         tokenized["labels"] = label
78         return tokenized
79
80     def __getitems__(self, indices):
81         return [self[i] for i in indices]
82
83 train_dataset = TokenizedDataset(trainset, tokenizer)
84 dev_dataset = TokenizedDataset(devset, tokenizer)
85
86 data_collator = DataCollatorWithPadding(
87     tokenizer=tokenizer,
88     padding="max_length",
89     max_length=config["max_length"],
90     pad_to_multiple_of=8
91 )
92
93 total_train_steps = len(train_dataset) // config["batch_size"] * config["
    epochs"]
94 warmup_steps = int(total_train_steps * 0.1)
95
96 training_args = TrainingArguments(
97     output_dir="./results2",
98     eval_strategy="epoch",
99     save_strategy="epoch",
100     per_device_train_batch_size=config["batch_size"],
101     per_device_eval_batch_size=config["batch_size"] * 2,
102     num_train_epochs=config["epochs"],
103     learning_rate=config["lr"],
104     weight_decay=0.01,
105     logging_dir="./logs2",
106     logging_steps=10,
107     remove_unused_columns=False,
108     label_names=["labels"],
109     bf16=True,
110     gradient_accumulation_steps=config["gradient_accumulation_steps"],
111     warmup_steps=warmup_steps,
112     lr_scheduler_type=config["scheduler"],
113 )
114
115 def compute_metrics(eval_pred):
116     predictions, labels = eval_pred

```

```
117     predictions = predictions.argmax(-1)
118     return {
119         "accuracy": accuracy_score(labels, predictions),
120         "f1": f1_score(labels, predictions, average="weighted"),
121     }
122
123 trainer = Trainer(
124     model=peft_model,
125     args=training_args,
126     train_dataset=train_dataset,
127     eval_dataset=dev_dataset,
128     data_collator=data_collator,
129     compute_metrics=compute_metrics,
130 )
131
132 trainer.train()
```

F Token Length Analysis

Listing 7: Data Analysis and Visualization

```
1 // DISCLAIMER: Code for this part is mainly written by Deepseek AI
2
3 import polars as pl
4 from transformers import AutoTokenizer
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 def analyze_token_lengths(file_paths, model_name="microsoft/deberta-v3-base"):
9     tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=False)
10
11     dfs = []
12     for path in file_paths:
13         try:
14             df = pl.read_csv(path, separator="\t", quote_char=None,
15                             has_header=True,
16                             new_columns=["id", "text", "label"])
17         except:
18             try:
19                 df = pl.read_csv(path, separator="\t", quote_char=None,
20                                 has_header=True,
21                                 new_columns=["text", "label"])
22                 df = df.with_columns(pl.lit("extra").alias("id"))
23             except Exception as e:
24                 print(f"Error reading {path}: {e}")
25                 continue
26         dfs.append(df)
27
28     all_texts = pl.concat([df.select("text") for df in dfs])
29
30     token_lengths = all_texts["text"].map_elements(
31         lambda x: len(tokenizer.tokenize(str(x))) if x else 0,
32         return_dtype=pl.UInt32
33     )
34
35     analysis_df = all_texts.with_columns(token_lengths.alias("token_length"))
36
37     stats = {
38         "num_samples": len(analysis_df),
39         "avg_length": analysis_df["token_length"].mean(),
40         "median_length": analysis_df["token_length"].median(),
41         "min_length": analysis_df["token_length"].min(),
42         "max_length": analysis_df["token_length"].max(),
43         "percentile_90": analysis_df["token_length"].quantile(0.9),
44         "percentile_95": analysis_df["token_length"].quantile(0.95),
45         "percentile_99": analysis_df["token_length"].quantile(0.99),
46     }
47
48     thresholds = [128, 256, 384, 512]
49     for threshold in thresholds:
50         count = analysis_df.filter(pl.col("token_length") > threshold).height
51         stats[f"over_{threshold}"] = count
52         stats[f"over_{threshold}_pct"] = (count / stats["num_samples"]) * 100
53
54     return stats, analysis_df
55
56 def print_stats(stats):
57     print("\nToken Length Statistics:")
58     for k, v in stats.items():
```

```

59         if isinstance(v, float):
60             print(f"{k:>20}: {v:.2f}")
61         else:
62             print(f"{k:>20}: {v}")
63
64     print("\nThreshold Exceedances:")
65     thresholds = [128, 256, 384, 512]
66     for t in thresholds:
67         print(f"{f'>{t} tokens':>20}: {stats[f'over_{t}']} samples ({stats[f'over_{t}_pct']:.2f}%)")
68
69 def plot_length_distribution(df, stats):
70     import seaborn as sns
71     import matplotlib.pyplot as plt
72
73     sns.set(style="whitegrid", context="talk")
74     lengths = df["token_length"].to_numpy()
75
76     plt.figure(figsize=(12, 6))
77     sns.kdeplot(lengths, fill=True, color="skyblue", linewidth=2)
78
79     thresholds = [128, 256, 384, 512]
80     colors = ["orange", "brown", "pink", "gray"]
81     for t, c in zip(thresholds, colors):
82         plt.axvline(t, color=c, linestyle=":", linewidth=1.5, label=f"{t} tokens")
83
84     metrics = [
85         ("Mean", stats["avg_length"], "red", "--"),
86         ("Median", stats["median_length"], "green", "--"),
87         ("95th %ile", stats["percentile_95"], "purple", "--"),
88         ("99th %ile", stats["percentile_99"], "darkblue", "--")
89     ]
90     for name, val, color, linestyle in metrics:
91         plt.axvline(val, color=color, linestyle=linestyle, linewidth=2, label=f"{name}: {val:.1f}")
92
93     plt.title("Token Length Density Plot", fontsize=18)
94     plt.xlabel("Token Length", fontsize=14)
95     plt.ylabel("Density", fontsize=14)
96     plt.xlim(0, 512)
97     plt.legend(loc="upper_right", fontsize=11)
98     plt.tight_layout()
99     plt.show()
100
101
102
103 if __name__ == "__main__":
104     file_paths = [
105         "../data/train.tsv",
106         "../data/dev.tsv",
107         "../data/extra.tsv"
108     ]
109
110     stats, df = analyze_token_lengths(file_paths)
111     print_stats(stats)
112     plot_length_distribution(df, stats)

```

G Additional preprocessing for Jigsaw dataset for Finnish model

Listing 8: Finnish preprocessing

```
1 import re
2
3 def extra_cleaning_jigsaw(text):
4     # 1) Remove table markup
5     text = re.sub(r'\{\|.*?\|\}', '', text, flags=re.DOTALL)
6
7     # 2) Remove template blocks
8     text = re.sub(r'\{\{.*?\}\}', '', text, flags=re.DOTALL)
9
10    # 3) Remove heading markup of the form == Some heading ==
11    text = re.sub(r'={2},.+?={2}', '', text)
12
13    # 4) Remove bracketed internal links or external links.
14    text = re.sub(r'\[[\].*?\]\]', '', text)
15    text = re.sub(r'\[http[^\]]*\]', '', text)
16
17    # 5) Remove bold/italic markers
18    text = re.sub(r"'''", '', text)
19    text = re.sub(r"''", '', text)
20
21    # 6) Remove any remaining HTML tags.
22    text = re.sub(r'<.*?>', '', text)
23
24    # 7) Remove style attributes or repeated quotes.
25    text = re.sub(r'style\s*=\s*"([^"]*)"','', text)
26    text = re.sub(r'"{2}', '', text)
27
28    # 8) Convert multiple newlines or spaces into a single space
29    text = text.replace('\n', ' ')
30    text = re.sub(r'\s+', ' ', text)
31
32    # 9) Strip leading/trailing whitespace
33    text = text.strip()
34    text = re.sub(r'^[\'\"\'s:]+', '', text, flags=re.MULTILINE)
35    return text
36
37 # Usage example
38 jigsaw_mt_data_train = jigsaw_mt_data_train.map(lambda x: {"text":
    extra_cleaning_jigsaw(x["text"])})
```